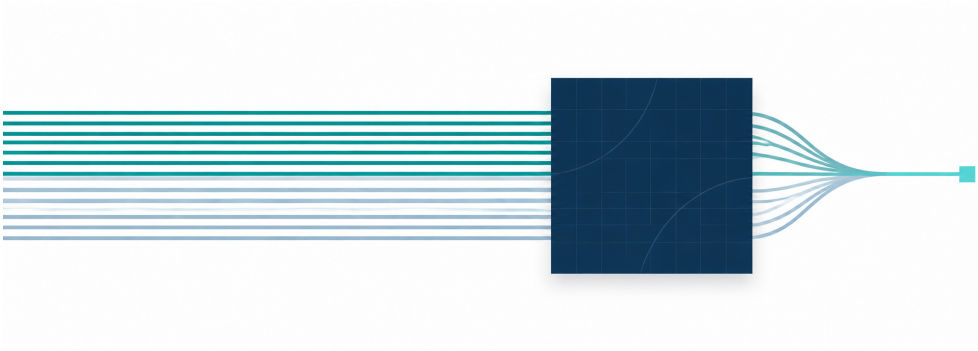
香山昆明湖 V2

自定义向量点积指令

第二阶段设计与基础测试文档

技术报告



AI 生成概念插图：不表示实际芯片版图或实测数据。

工程组合： 0f ＋ firstdot V2 ＋原 C8

团队名称： 待团队确认

成员： 待团队确认

学校： 待团队确认

材料日期： 2026 年 11 月 2 日

图像修订： 2026 年 11 月 5 日

已补齐原 9 处空图位；身份信息与波形补证范围见正文。

目录

第二阶段设计与基础测试文档 1

2.1 设计目标 1

2.2 指令语义与编码 1

2.3 数据通路和两槽微架构 2

2.4 与 DIV 共用父级的处理 3

2.5 功能验证 4

2.6 设计取舍与扩展 6

2.7 工程实现与验证方法补充 6

2.7.1 从数学定义到硬件位宽 6
2.7.2 译码、微操作和配置依赖 6
2.7.3 P 槽、R 槽与同拍重叠 7
2.7.4 反压期间的身份稳定性 8
2.7.5 取消、复位及环形年龄 9
2.7.6 共享 DIV 资源与时钟使能 10
2.7.7 定向、随机和全核测试的分工 11
2.7.8 设计演进与最终采用 12
2.7.9 指令语义与实现边界的逐层核对 13
2.7.10 算术边界为何需要单独覆盖 13
2.7.11 配置依赖和写回路径的风险 14
2.7.12 两级弹性管线的状态推导 14
2.7.13 父级共享资源与门控检查 15
2.7.14 测试证据如何形成可复核结论 15
2.7.15 优化筛选与交付判据 16
2.7.16 回归矩阵中的组合关系 16
2.7.17 从日志到源码的审查步骤 17
2.7.18 时序与吞吐取舍的解释 17
2.7.19 面向下一阶段的接口冻结 17

2.7.20 回归失败的归属与收口 18
2.7.21 写回身份与功能结果的联合检查 18

2.8 内部 ISA 合同、数学边界和测试向量 19
2.8.1 数学语义与目的位布局 19
2.8.2 合法配置、别名、异常与状态 20
2.8.3 候选机器编码 22
2.8.4 RTL、NEMU 与验证共同遵守的接入要求 23
2.8.5 测试与验收合同 25
2.8.6 RVV 性能基线必须保持的等价性 28
2.8.7 歧义清单与变更流程 28

2.9 两槽弹性状态、父级协议及验证不变量 29
2.9.1 接口与状态 29
2.9.2 每拍转移定义 30
2.9.3 父级资源、派发与除法切换 32
2.9.4 门控、唤醒与写回 33
2.9.5 观察协议与不变量 33
2.9.6 分阶段验收与负控 34

2.10 本轮已提供截图补充 35

第二阶段设计与基础测试文档

工程组合：0f ＋ firstdot V2 ＋原 C8　材料日期：2026 年 11 月 2 日

工程采用 0f 硬件，材料版本 V1.0，初稿日期为 2026 年 11 月 2 日。

2.1　设计目标

点积由多次乘法与累加构成。本设计在昆明湖 V2 已有向量路径中增加 vdot 功能，复用处理器的译码、向量寄存器、调度、写回和精确提交机制。优化集中于独立点积吞吐、结果保存成本及与 DIV 共用父级时的取消行为。

2.2　指令语义与编码

内部合同 vdot-s8x8-r32-v0.1 固定 VLEN128、SEW8、LMUL1、VL8 及 vstart0。两个源各取低 8 个字节，按 signed int8 解释。

$S = \Sigma_{(i=0\cdots7)}\ \text{signed8}(vs1[i]) \times \text{signed8}(vs2[i])$

目的低 32 位保存 S 的二补码，高 96 位清零。源高 64 位不参与计算。结果覆盖旧目的，源与目的允许别名。单项乘积范围为 −16256～16384，总和范围为 −130048～131072，32 位结果能够完整保存。

表 2-1　指令语义与编码

字段	位段	取值
funct6	31～26	000000
vm	25	1
vs2 / vs1	24～20 / 19～15	向量源寄存器编号
funct3	14～12	000
vd	11～7	向量目的寄存器编号
opcode	6～0	custom-0，0x0b

MATCH 为 0x0200000b，MASK 为 0xfe00707f。汇编记法使用 vdot.vv vd, vs2, vs1，程序通过校验后的机器字/汇编宏编码。示例 vdot.vv v10,v8,v9 对应 0x0284850b。

合法状态还要求 vill=0 和 vm=1，接受四种 vta/vma 组合。非法配置通过异常路径处理，成功运算保留合同规定的向量 CSR。对目的宽度、合法域等赛题歧义，按内部合同公开说明，并保留 Q01～Q07 待官方澄清状态。



图 2-1　vdot 指令字段与完整结果布局

注：两个源寄存器均只读取低 8 个字节，按 signed int8 相乘并求和；完整 128 位目的结果的低 32 位为补码结果，高 96 位清零。

来源与范围：用户提供示意图；依据：固定 team/spec/vdot.md 及本文编码表。版本范围：内部 vdot-s8x8-r32-v0.1 合同；保留歧义说明。

2.3　数据通路和两槽微架构

数据通路由 8 路有符号乘法、乘积寄存器、归约加法网络及结果槽构成。乘积为 16 位有符号数，归约在足够宽度下累加。

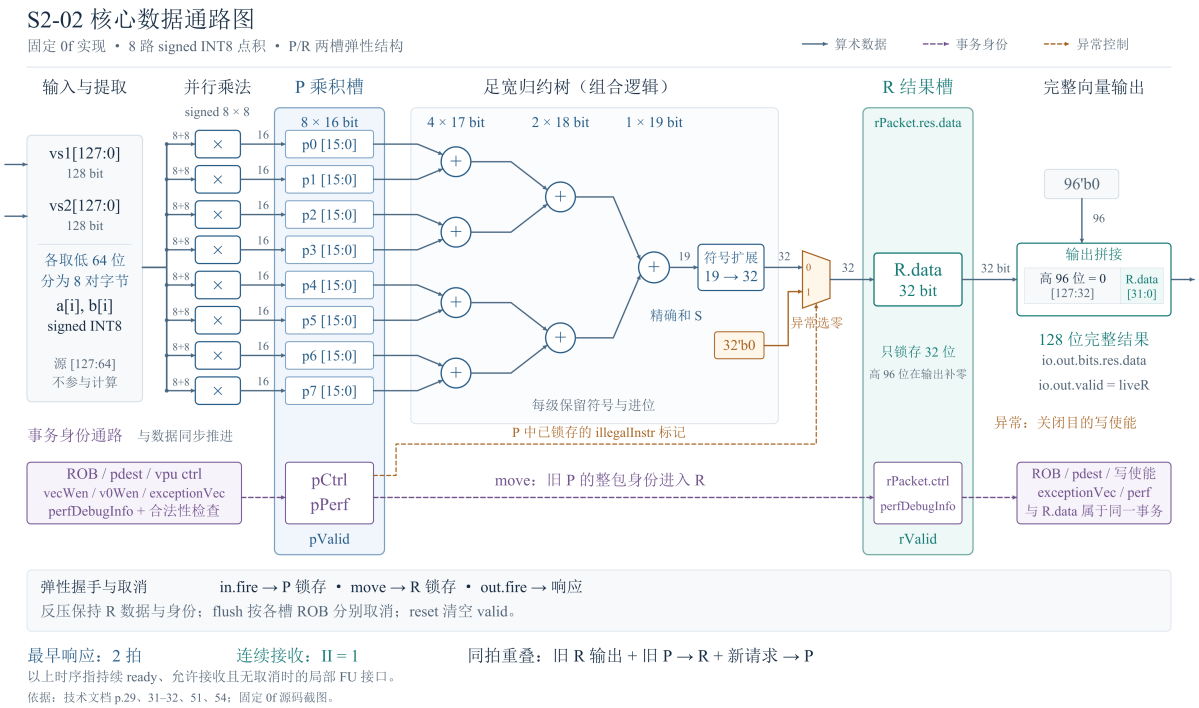


图 2-2 0f 点积算术与 P/R 两槽数据通路

注：8 路乘积经 P 槽、足宽归约及 R 槽输出；ROB、pdest、控制和性能标记随事务传递。图示为稳定 0f 两槽实现，无阻塞时最早 2 拍响应，启动间隔为 1 拍。

来源与范围：用户提供示意图；依据：0f 固定提交的 VDotElastic/VDotO2 及算术模块；旧 TikZ 图可参考。版本范围：最终采用 0f；不展示未采用 P4 一拍候选。

P 槽保存 8 个乘积及对应控制信息，R 槽保存完成结果和完整身份。无阻塞时，旧 R 输出、旧 P 转入 R、新请求进入 P 可在同一拍发生。最早响应为接收后 2 拍，连续接受间隔为 1 拍。

H1 将结果状态保存宽度收窄到 32 位，输出处拼接 96 个零。ROB 标识、目的物理寄存器、向量写使能和性能信息随事务一起推进，反压期间保持待输出数据及身份。flush 分别匹配 P、R 的 ROB 标识，处理部分取消与同时取消。异常请求携带异常信息并关闭目的写使能。

实现路径包括 backend/fu/vector 中的算术模块、wrapper/VDotElastic.scala 及 VDotO2.scala，父级调度和时钟控制位于 backend/exu/ExeUnit.scala。实际浏览以固定 0f 提交的路径为准。

2.4 与 DIV 共用父级的处理

VFEX4 在 vdot 及整数/浮点 DIV 之间管理分派、占用和时钟。0f 修复匹配 DIV 取消之后保持状态未及时释放的问题。验证通过实际父级信号检查取消后的空闲时钟，并覆盖复位、两类 DIV、反压和请求切换。两槽占用信息用于保持正确互斥，控制信号以实际事务身份为依据。

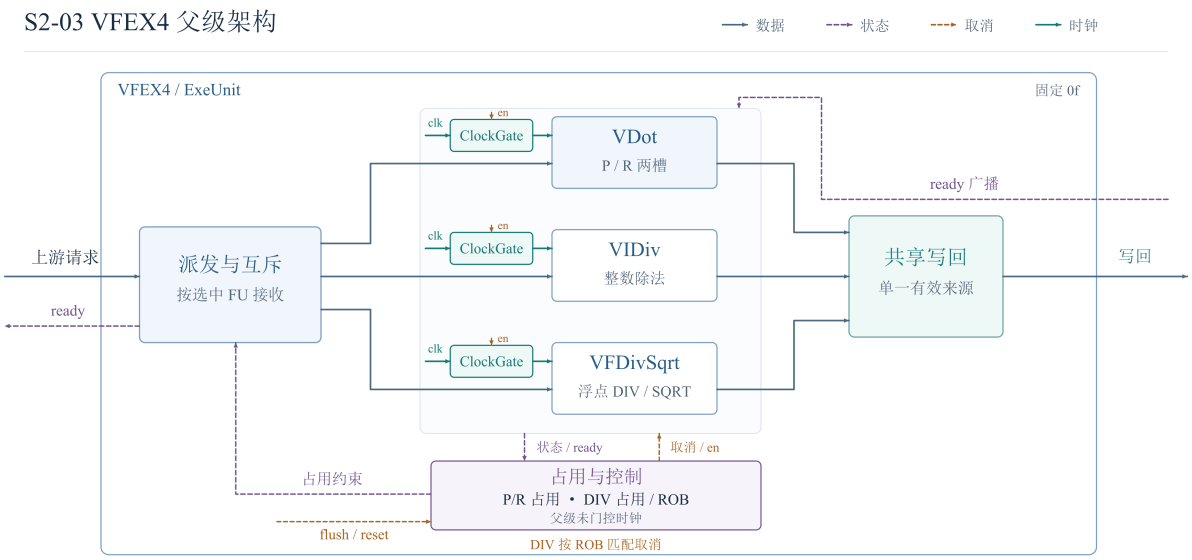


图 2-3 VFEX4 父级的派发、互斥与门控结构

注：VFEX4 共享资源的简化结构示意图，展示 VDot、VIDiv、VfDivSqrt、门控及共享写回之间的关系；控制细节以稳定 0f 实现和正文说明为准。

来源与范围：用户提供示意图；依据：0f ExeUnit.scala、父级验证及 DIV 取消修复证据。版本范围：0f 生产逻辑；不得套用未采用 C1/C2/C3/P4。

2.5 功能验证

测试从独立数学模型推进到算术 RTL、FU、真实父级和全核差分。各层统计分别保存。

表 2-2 功能验证

层级	主要覆盖	已有结果
模型/算术	22 组定向＋ 10000 组随机，完整 128 位	验证通过
FU/真实父级	反压、取消、复位、身份、异常及 DIV 交互	0f 原证据复用，4 ＋ 16 项负控
全核	正负边界、寄存器别名、非法向量状态等	117 个不同用例，按原矩阵分片执行 124 次
总验收	原 13 项与补 1 项组合、负控及恢复	R5/G5 组合验证通过

下面列出便于评审阅读的定向预期值，完整执行证据见原测试归档。

表 2-3 功能验证

输入	数学结果	目的低 32 位
[1,2,3,4,5,6,7,8] 与自身点积	204	0x000000cc
8 个 −1 与 8 个 1 点积	−8	0xfffffff8
8 个 −128 与 8 个 −128 点积	131072	0x00020000
8 个 −128 与 8 个 127 点积	−130048	0xfffe0400
8 个 0 与任意合法输入	0	0x00000000

所有行还需检查高 96 位为零。测试对源高半区和旧目的设置哨兵，以发现错误 lane 选择、目的高位保留和源/目的别名处理问题。

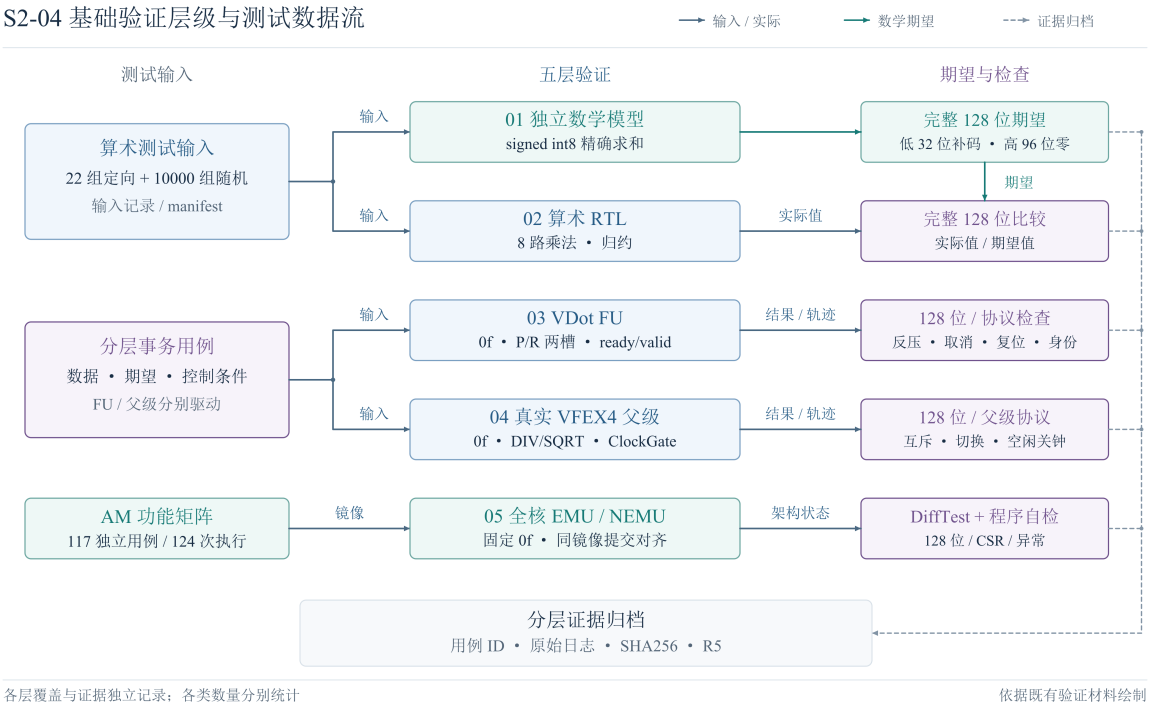


图 2-4 基础验证层级与测试数据流

注：独立参考模型、算术 RTL、FU、真实 VFEX4 父级和全核 DiffTest 分别核验；22 定向加 10000 随机属于算术层，117 独立用例、124 次执行属于所列全核范围。

来源与范围：用户提供示意图；依据：基础测试矩阵、模型数据 manifest、R5 验收矩阵。版本范围：各层覆盖独立记账；Agent 检查不标成人工验收。

2.6 设计取舍与扩展

O2 以两槽容量提高独立吞吐，H1 降低结果寄存器开销，0f 完善 DIV 取消关钟行为。后续一拍 P4 候选经历局部验证和成本筛选，原版 6 项、唯一修订 10 项门槛未过，最终选择 0f。

参数扩展实验另行实现 signed INT16×8、64 位结果，完成 94 组定向和 10000 组随机验证。该证据说明算术与包装的参数化方法，生产实现的 ISA 域按 INT8 合同交付。

指令合同 · 稳定 FU 源码 · R5 验收矩阵 · 参数扩展

2.7 工程实现与验证方法补充

2.7.1 从数学定义到硬件位宽

signed int8 的范围为 −128 至 127。最大正乘积由 −128 乘 −128 得到 16384，最负乘积由 −128 乘 127 得到 −16256。八项累加后的数学范围为 −130048 至 131072。19 位有符号表示覆盖该范围，18 位正上限只有 131071，因此正端点是检验归约位宽的必要案例。

加法树每一级保留进位，宽度依次由 16 增加到 17、18、19 位。最终先形成 32 位补码，再在外部拼接 96 位零。负数结果的高 96 位仍为零，因为项目将低 32 位定义为结果字段，整个 128 位向量按照字段布局写回。完整 128 位检查能够同时发现符号扩展方向错误和高位残留。

源寄存器高 64 位设置非零哨兵，可以检验实现是否误做 16 项点积。两个源的字段提取还需要独立核对，因为点积交换律会掩盖 vs1 与 vs2 交换的问题。数学结果正确、编码字段正确、实际寄存器依赖正确是三项分别检查的条件。

2.7.2 译码、微操作和配置依赖

精确编码匹配后，指令应进入向量类别并生成单个微操作、单次向量写回。编码中寄存器字段采用与 R 型相似的位置，但寄存器类别属于向量体系。若将字段直接按整数寄存器读取，简单数值案例可能偶然通过，架构状态和别名行为仍会错误。

实际 VL 通过重命名后的依赖到达执行端。SEW、LMUL、vill 等来自随指令携带的向量控制。对于乱序处理器，执行时读取一个全局当前 CSR 无法保证对应到本条指令的配置，尤其当多条 vset 与向量运算交错时。测试因此包含密集配置变化、合法非法交错和恢复。

旧目的寄存器不参与点积累计，译码需要避免无用旧目的依赖。目的可能是 v0，也可能与某个源相同，因此源快照和目的写回类型必须明确。完成标记、异常信息和写使能与这一次微操作绑定，防止推测执行在取消后留下架构可见更新。

2.7.3 P 槽、R 槽与同拍重叠

P 槽包含乘积和完整身份，R 槽包含归约结果和完整身份。两个 valid 描述各槽是否拥有有效事务，而不是描述算术结果是否非零。结果为零的合法事务仍必须完成一次响应。

设旧 P 为 A、旧 R 为 B。当下游 ready 且有新请求 C 时，当前边沿可以同时完成 B 输出、A 从 P 转入 R、C 进入 P。非阻塞赋值和实际组合逻辑必须读取旧 P 的内容，使 A 的结果不会被 C 覆盖。验证通过相异数据及相异 ROB/pdest 组合来识别这类覆盖错误。

当 R 被反压时，P 可在空槽条件下接收一个请求，但满容量后必须停止接收。反压解除时，R 响应与 P 推进可以重叠，避免恢复过程引入额外气泡。每次握手、推进和取消均在同一全局周期坐标中复算，不能从日志打印顺序推断真实边沿顺序。

表 2-4 P 槽、R 槽与同拍重叠

旧 P	旧 R	下游 ready	可以发生的主要动作
空	空	任意	接收新请求到 P
有效	空	任意	旧 P 推进到 R，并可接收新请求
空	有效	0	保持 R，可接收新请求到 P
有效	有效	0	保持两槽并反压输入
有效	有效	1	R 输出、P 推进、新输入接收重叠

上表说明正常流的容量关系。reset 与 redirect 具有额外优先级，必须在完整状态转移中覆盖。

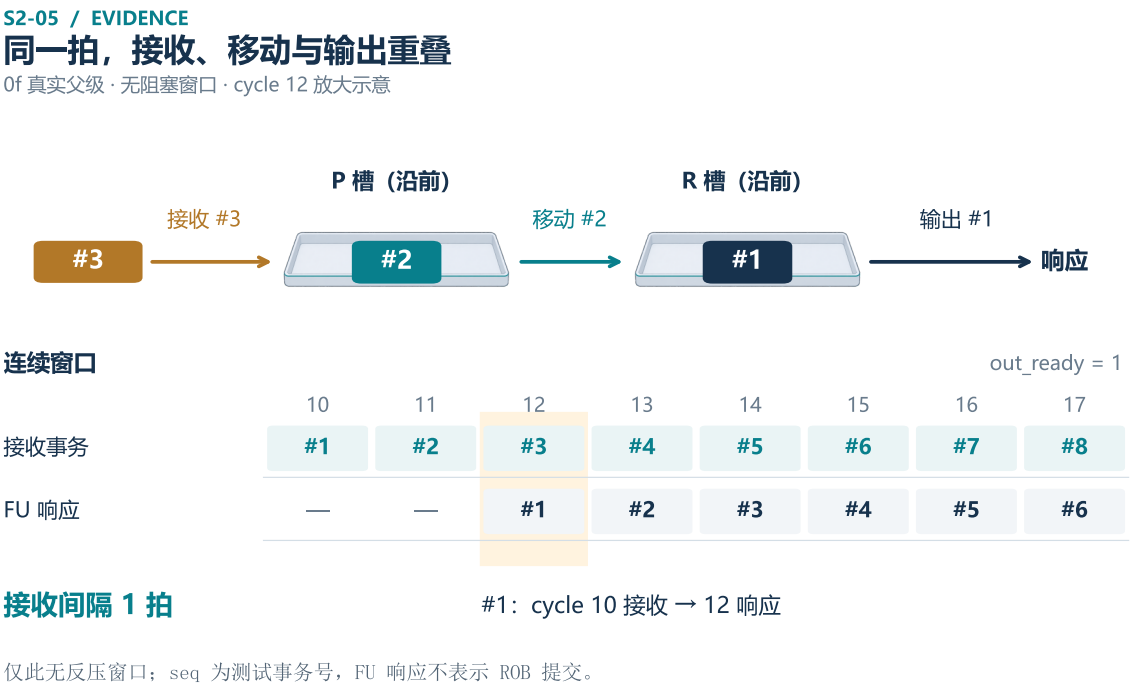


图 2-5　无阻塞连续接收的两槽重叠事件

注：cycle 12 中旧 R 输出、旧 P 转 R 和新请求接收同拍发生；无阻塞时事务 1 在 cycle 10 接收、12 响应，连续接收间隔为 1 拍。seq 为测试记分板事务号，图中槽状态为沿前采样。

来源与范围：0f 真实 VFEX4 父级 green-parent-01/result.log.gz，原行 12–19；case=frozen-vdot-II1。版本范围：历史父级逐拍记录；响应为 FU 握手而非 ROB 提交；未伪造时钟或完整数据总线波形。

2.7.4　反压期间的身份稳定性

io.out.valid 表示 R 中存在可输出事务，其含义独立于 io.out.ready。输出被阻塞时，结果、ROB 标识、目的物理寄存器、异常向量、写使能及性能字段均保持。只检查 data 稳定而忽略控制包，可能产生数值正确但写入错误寄存器的问题。

验证为每个接受事务记录独立身份，响应时与该身份的参考值对应。负控包括交换两事务 pdest、改变 ROB、篡改高位、制造重复响应或漏响应。实际硬件协议测试与检查器日志变异分别记账，前者验证设计，后者验证证据审查能力。



图 2-6 反压期间完整结果与事务身份保持

注：同一事务 seq1321 于 181597 接收，181599–181605 连续 7 拍反压，完整 128 位结果 0x00000000000000000000000000001A87 及 ROB/pdest/控制字段保持；181606 只有一次 FU 输出握手。图中分别保留父级 inready 与 FU vready；181606 后者恢复而前者仍为 0，同拍旧 P 事务 1322 转入 R。

来源与范围：0f fullcore/core-01/control/run.log.gz，原行 15012–15044；raw SHA256 c7351619b11e61171009c1dc997c08dd8a575f7c1508d79f7484df72711c6cb7。版本范围：历史 EMU 2d1db77cc407…，非 clean-02 acf672…；接收到响应 9 拍含 7 拍反压，不是 ROB 提交或新仿真。

2.7.5 取消、复位及环形年龄

redirect 按 RobPtr 的年龄语义分别判断 P 和 R 是否需要取消。ROB 索引的数值在环回时不能直接代表年龄，测试需要覆盖 flag 变化、不同 flushItself 设置和接受顺序与年龄关系不同的情况。较老幸存事务继续完成，较年轻匹配事务释放有效状态。

同一拍发生输出 ready 和匹配取消时，取消决定该事务是否仍可形成响应。reset 则清除槽有效状态，并检查接口握手被抑制。真实父级还需验证已经关钟的空槽可以正确复位，以及取消后重新唤醒的第一笔请求不丢失。

任意前缀都满足事务守恒。接受数等于已响应数、已取消数、复位丢弃数和当前在途数之和。当前在途数最大为 2。这个关系能够发现尾部漏计、幽灵响应和被取消事务再次输出等错误，并要求正常结束时排空。

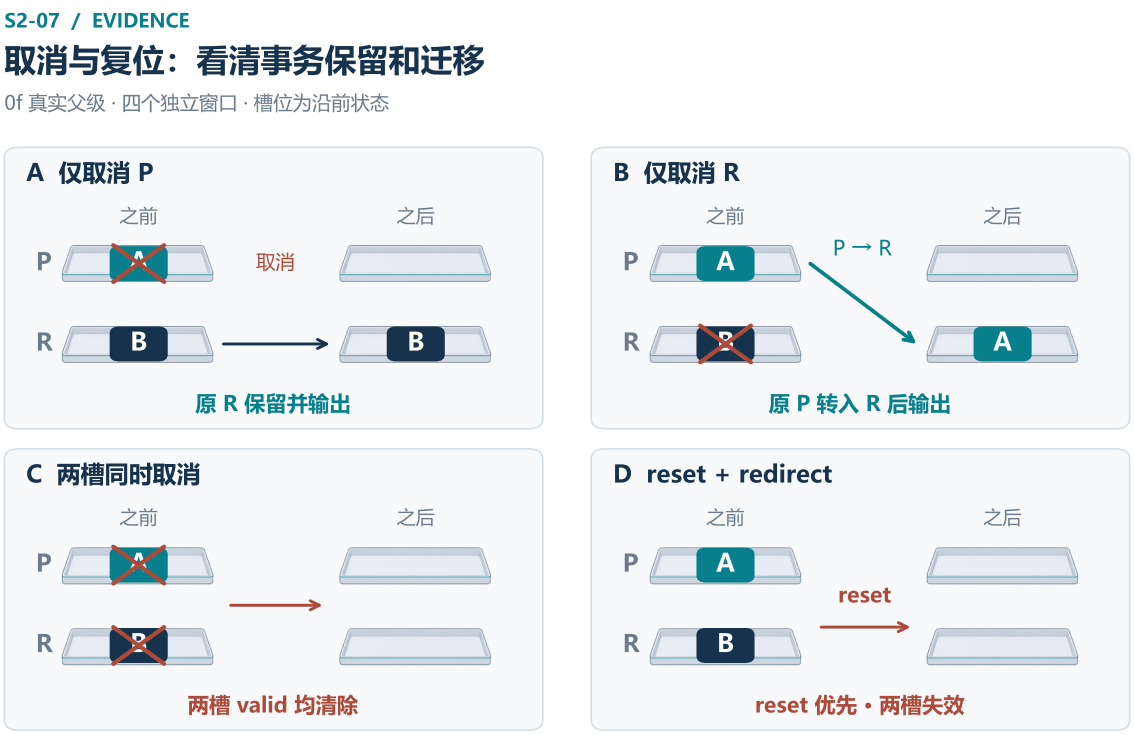


图 2-7　部分取消、全部取消与复位优先级

注：四个独立窗口展示仅 P 取消、仅 R 取消且 P 转 R、两槽取消，以及 reset 与 redirect 同拍时按 reset 唯一记账。前两例包含 ROB flag 环绕；seq 不能代替程序年龄。状态按各窗口沿前记录抽象展示，原 cycle 及事务身份见附件；空槽仅表示 valid 清除。

来源与范围：green-parent-01/result.log.gz；partial-ROB-48/54/12 及 TE0-reset-7；原行与逐行值见 figures/evidence-20261105/figure-data.json。版本范围：仅 0f 真实父级已记录窗口；被取消请求无错误响应，幸存请求按原身份输出。

2.7.6　共享 DIV 资源与时钟使能

VFEX4 父级在 vdot 和 DIV 之间保持资源互斥，输出选择要求有效完成来源唯一。仅在独立 FU 中验证 ready/valid，无法证明父级在取消和切换时正确释放 busy 和时钟保持状态。最终动态父级验证包含 VDot、VIDiv、浮点 DIV 以及真实 ClockGate；SQRT 取消路径完成静态检查，不能将 DIV 动态记录扩展为 SQRT 动态覆盖。图中子时钟位掩码来自上升沿后采样，不是门控使能波形，也不直接代表功耗。

0f 处理的是匹配 DIV 取消后保持状态的释放。若取消后仍保持时钟使能，空闲期间会观察到多余时钟脉冲。若过早释放，又可能使仍在途的事务冻结。因此测试同时核对取消后的空闲采样和未取消事务的完成行为。



图 2-8　DIV 切换与取消后关钟

注：上部展示先排空 VDot 再接收整数/浮点 DIV；下部展示取消清理边沿后子时钟归零，再接收新 VDot。clocks 为上升沿后采样的子时钟位掩码，1/2/4 分别表示 VDot/VIDiv/VFDiv，不是 gate_enable 信号。

来源与范围：green-parent-01/result.log.gz；switch-0-1-0、switch-0-2-0、DIV-cancel-92/110；完整行号见图数据附件。版本范围：动态覆盖 DIV；SQRT 仅静态取消路径检查。事件箭头不表示时间比例；不宣称功耗测量。

2.7.7　定向、随机和全核测试的分工

定向测试用最小数据隔离某一类错误，随机测试扩大数据覆盖并检查确定性再生成，全核测试验证实际译码、依赖、异常和提交路径。完整 128 位数学比较贯穿各层，而配置、取消和写回身份需在对应控制层检查。

22 组定向和 10000 组随机描述算术层输入集合。117 个不同功能/异常例描述架构测试矩阵，124 次执行包含分片重复对照。这些计数保持各自单位，避免把重复执行累计为新的不同用例。

基础测试报告应对每类输入说明初态、配置、指令字、预期结果、实际检查域和通过条件。负数案例核查低 32 位补码与高 96 位零，极值案例核查位宽正端点，别名案例核查源快照与目的分配，非法配置案例核查精确异常和写入抑制。

(a) Historical combinational RTL suite counts (2026-10-12)

```
VDOT_DATAPATH_SUITE_PASS suite=directed count=22
VDOT_DATAPATH_SUITE_PASS suite=random count=10000
VDOT_DATAPATH_PASS directed=22 random=10000 total=10022
```

(b) Positive transaction: expected 204 and actual 128-bit response

```
[POSITIVE] functional-2 seq=1 ROB=0:39 pdest=103
vs1 (128-bit)     0xa5a5a5a5a5a5a5a50807060504030201
vs2 (128-bit)     0xa5a5a5a5a5a5a5a50807060504030201
expected sum      204
expected (128-bit) 0x000000000000000000000000000000cc
actual  (128-bit) 0x000000000000000000000000000000cc
ACCEPT cycle=339296 line=7030 | RESPONSE cycle=339298 line=7034
source: clean-02/fullcore-run-01/runs/functional-2/run.log
SHA256: 90b36b19e9d141e1b0bdad6bed461b4dbde4e534798f5fbdaa8819d9e2ef3211
```

(c) Negative transaction: expected -8 and actual 128-bit response

```
[NEGATIVE] functional-0 seq=4 ROB=0:12 pdest=143
vs1 (128-bit)     0xa5a5a5a5a5a5a5a5ffffffffffffffff
vs2 (128-bit)     0xa5a5a5a5a5a5a5a50101010101010101
expected sum      -8
expected (128-bit) 0x000000000000000000000000fffffff8
actual  (128-bit) 0x000000000000000000000000fffffff8
ACCEPT cycle=420291 line=7927 | RESPONSE cycle=420293 line=7931
source: clean-02/fullcore-run-01/runs/functional-0/run.log
SHA256: 5bd56636056d7b8c77ab976511522c6654c112644f0ea024dfac99bb92f855f7
```

(d) Boundary transaction: expected 131072 and actual 128-bit response

```
[BOUNDARY] functional-0 seq=5 ROB=0:43 pdest=66
vs1 (128-bit)     0xa5a5a5a5a5a5a5a580808080808080
vs2 (128-bit)     0xa5a5a5a5a5a5a5a580808080808080
expected sum      131072
expected (128-bit) 0x00000000000000000000000000020000
actual  (128-bit) 0x00000000000000000000000000020000
ACCEPT cycle=448417 line=8400 | RESPONSE cycle=448419 line=8404
source: clean-02/fullcore-run-01/runs/functional-0/run.log
SHA256: 5bd56636056d7b8c77ab976511522c6654c112644f0ea024dfac99bb92f855f7
```

图 2-9　完整 128 位算术结果证据（异常截图待补）

注：(a) 组合算术 RTL 统计；(b)-(d) 历史全核观察器 ACCEPT/RESPONSE 配对的字段摘录，按 seq、ROB 和 pdest 绑定。actual 来自原响应日志，expected 按低 8 个 signed int8 重新计算；不是 GTKWave 波形或架构提交截图。非法配置异常码和目的不更新的图片仍待补。

来源与范围：用户提供的算术统计截图及正数、负数、边界响应摘录截图；图中保留原日志路径、行号和 SHA。版本范围：算术 RTL 统计为 2026-10-12 历史记录；0f 全核响应为 2026-10-27 UTC 历史记录；原批次 UTC 容量审计 FAIL 保留。

2.7.8　设计演进与最终采用

初版单请求控制提供功能闭环，T14 改变乘积锁存位置，O1 支持更及时的交接，O2 通过两个弹性槽支持连续接受。H1 将结果状态收窄至 32 位，0f 进一步完善 DIV 取消行为。每项演进都对应其独立的验证和比较合同。

后续 P4 尝试将内部算术缩为一拍，局部验证完成后进入同约束成本筛选。原 47 项门槛中，初版失败 6 项、唯一修订失败 10 项，因而最终报告仍采用 0f。设计决策同时考虑协议、功能、资源与时序，单一指标改善不足以替代完整资格。

2.7.9 指令语义与实现边界的逐层核对

第二阶段的设计审查从软件可见的结果开始，逐层追到硬件内部状态。软件合同首先规定指令从两个向量源读取低位的八个有符号字节，逐项相乘并求和，形成低三十二位结果，同时将结果向量其余九十六位清零。这个定义决定了参考模型、定向数据和写回比较的共同口径。指令编码、寄存器字段和向量配置则决定该结果是否属于当前指令。即使算术值相同，若译码选择了另一类功能单元，或输出进入错误的物理目的寄存器，架构行为仍然失败。

评审时需要区分项目约定和外部通用向量扩展的语义。这里的点积是项目实现的定制指令，测试中的字段布局、异常条件和目标寄存器宽度应以本项目已冻结的输入合同为准。不能因为指令名字中含有向量术语，就自动把标准向量指令所有尾部策略、掩码策略或多元素迭代规则套进该实现。相反，也不能用局部算术单元测试的成功替代对项目译码约定的检查。报告在每一层标出该层能够证明的内容，防止一份波形被重复解释为全链路证据。

完整输入合同应包含指令字、两个源寄存器的全部一百二十八位、目的寄存器编号、向量控制快照、进入功能单元时的事务身份，以及可能同时存在的取消信息。期望输出除了结果数据，还包含有效位、目的物理寄存器、ROB 身份、异常字段和写回使能。这样描述允许复核者从测试向量直接推导预期，而不必依赖某一个实现版本的内部信号名。字段无法由输入推导时，应在测试中明确标成由父级供应的控制元数据。

2.7.10 算术边界为何需要单独覆盖

正端点一三一零七二来自八次负一百二十八乘负一百二十八，恰好越过十八位有符号数的上限。若硬件在归约树末级之前只保留十八位，常规随机样本仍可能大量通过，而这一个构造样本会确定性失败。负端点来自八次负一百二十八乘一百二十七，可检查负数中间值和最终补码。零、正一、负一及交错正负项则分别检查零传播、符号处理和加法顺序。这些用例不是为了增加表格数量，而是分别锁定不同的失效机制。

乘积的符号扩展必须在参与加法之前完成。对有符号八位源直接截取无符号字段，再在加法树末端补符号，不能恢复正确的中间结果。各级加法树的位宽应按数学范围增加，并在每一级检查进位和符号位。最终低三十二位的形成方式与整条向量的高位清零是两个独立操作；两者同时检查，可以区分加法位宽错误、错误的整向量符号扩展和旧目的数据残留。

源寄存器的高半区作为哨兵时，应选择能显著改变错误十六项实现结果的非零数据。只把高半区填成零虽然满足合法输入，却无法识别错误的读取长度。同样，交换两个源所得的数学点积不变，所以对源顺序的检查应依赖源读取使能、寄存器依赖和已知非交换的控制情境，而不能只看最终和。测试把可观测的算术行为与必须在译码或重命名处观察的行为分开，避免把交换律造成的盲点当成覆盖。

2.7.11 配置依赖和写回路径的风险

乱序核心中，相邻指令进入执行单元的时间可能与程序次序不同。若点积指令执行时再读取一个可变的全局 VL 寄存器，就可能得到后来配置指令的值。正确的验证对象是随本条微操作传播的配置依赖以及它与源数据的配对关系。测试用合法配置、非法配置、短间隔切换和取消后的重新配置，分别观察译码、调度和执行端是否维护同一份指令上下文。对于没有实现的配置组合，应检查明确拒绝及异常抑制，而不是让它们悄悄落入默认算术路径。

源寄存器与目的寄存器编号相同，是检验重命名和源快照的有效情境。接受请求时，源值应已固定为该请求的操作数；之后目的写回不得反过来改变在途请求所用的源。目的为零号向量寄存器时，也要依照项目合同检查它是合法的目的字段还是特定控制字段，不能套用整数零寄存器的丢弃规则。只有从译码类型、读端口、执行响应到提交映射均保持一致，测试才可称为覆盖完整写回路径。

异常和取消是写回路径的一部分。非法编码、非法向量配置或父级产生的异常需要与原始事务身份同行，并禁止正常结果在取消后写回。单看算术结果端口会漏掉这类问题，所以检查器同时记录发射、接受、响应和提交观察点。每个观察点只对本层可见的字段负责，之后以身份键将这些记录拼接，核对是否存在错配、重复或消失的事务。该方法也方便将未来新增的异常类型接入同一套审查框架。

2.7.12 两级弹性管线的状态推导

P 槽负责保存尚待归约的乘积与身份，R 槽负责保存已归约的结果与身份。两槽各自拥有有效位，数据寄存器里保留旧值并不表示存在有效事务。计算输入 ready 时，需要综合 P 槽是否空闲、P 能否在该边沿前进、R 是否空闲以及 R 能否在该边沿输出。计算输出 valid 时，则直接以旧 R 有效位为准。把 ready 作为 valid 的条件会在反压时隐藏一个真实存在的响应，造成接口协议错误。

一个周期内可以同时发生三个动作，但动作操作的是三个不同时间层的对象。旧 R 可能响应，旧 P 可能进入新 R，新输入可能进入新 P。审查组合逻辑时，应画出边沿前的状态和边沿后的状态，而不是按源码赋值顺序想象三个顺序执行的步骤。特别是新输入覆盖 P 寄存器的同一边沿，R 所接收的必须仍是旧 P 的乘积。用不同的结果与不同的目的寄存器交错输入，可以直接暴露把新 P 误送进 R 的错误。

容量为二的结论需要纳入取消和复位的额外状态转移。若 R 反压而 P 空闲，允许填充 P；若两槽都满，则输入必须停止。如果 redirect 只取消 R 而保留 P，下一边沿可以由幸存 P 填补 R；若只取消 P，R 可继续等待正常响应。reset 清空两个有效位后，旧寄存器中的比特无须立即清零，但它们不得再次获得有效身份。状态表和守恒关系共同检查这些情况，比只观察几条理想流量波形更可靠。

ROB 环形指针具有索引和环回标志。数值较小的索引在环回之后可能代表较新的事务，因此年龄判断必须使用父级定义的比较语义。定向取消样本要跨过环形边界，并

分别覆盖 flushItself 启用和关闭。测试还需让 P 与 R 的 ROB 年龄和槽位先后关系不完全一致，以防代码误把槽位置当作年龄。对每种情况，先列出应幸存的身份集合，再检查输出序列是否恰好等于该集合。

2.7.13 父级共享资源与门控检查

点积加入 VFEX4 后，父级同时承载除法等已有操作，因而必须检查不同来源的响应选择和 busy 释放。若点积自己的两槽逻辑正确，父级仍可能在 DIV 取消后留下旧 busy，阻断新的点积请求；也可能提前放开共享资源，使两个来源在同一输出端口争用。父级验证使用交替操作、同时反压和匹配取消来观察这些问题，且明确记录输出选路所采用的事务身份。

真实时钟门控单元还引入唤醒边界。门控使能往往根据当前有效状态和新请求组合得出；取消、复位、测试使能与时钟相位的组合可能产生独立仿真里没有的差异。对门控的检查不能只比较最终数值，还要观察空闲时是否出现不需要的时钟活动，以及唤醒的第一笔请求是否完整进入 P 槽。对取消 DIV 的修订也应在同一个父级环境中验证，避免只在点积专用平台上得到局部成功。

仓库中的 O1 与 O2 默认开关保持关闭，避免常规构建无意切换实验实现；本报告选定的 0f 稳定构建则按 clean-02 冻结记录显式启用 O2/T14 并关闭 O1。两种配置不能共用同一性能数字。候选比较必须保存开关、源码与构建摘要，在相同输入和约束下复验算术、取消、门控及父级路径；未通过完整门槛的结果仍标为实验结果。由此可以分别追溯默认构建、正式 0f 构建和其他候选的资格范围。

2.7.14 测试证据如何形成可复核结论

算术定向用例提供对边界和特殊值的解释，随机用例提供更宽的输入分布，父级定向用例提供协议和共享资源证据，全核用例提供架构路径证据。每层使用同一数学参考值，但观察对象不同。随机种子、用例数量、构建标识和检查器版本需要随日志保留；否则一个“全部通过”的汇总行无法说明实际跑过哪一批输入。重新运行时还应核对生成输入与原始日志记录一致。

负控的作用是证明检查器会报错。修改期望结果高位、交换两个事务身份或删除一条响应时，检查器应针对相应字段失败。负控本身并不证明硬件存在这些错误，也不能计入硬件通过数。它限定的是证据系统的灵敏度：如果某种故障注入后仍然显示通过，原本的正向测试即使全部绿色，也缺乏针对该故障的检出能力。报告把两类运行分别标识，便于审阅者正确使用数字。

全核验证要包含真正的译码、配置依赖、读端口、执行单元、写回和提交。只在测试平台直接驱动 VDot 请求不能覆盖编码匹配错误；只读取最终软件寄存器又可能难以定位响应身份问题。因此定向平台与软件用例相互补充。对于尚未取得完整输入合同的分支，应记录缺少哪些构建物、指令字或配置，避免把无法复现解释成实现已经通过。

阶段结论建立在可重新执行的输入和现有日志上，而非口头推断。

2.7.15 优化筛选与交付判据

性能筛选要求候选版本在相同约束、相同工作负载和相同观察窗口下比较。两级弹性管线改善的是连续接受与反压恢复能力，不直接保证单条指令延迟、面积或最高频率也改善。H1 收窄状态位宽主要影响寄存器规模；门控修订主要解决空闲切换和取消后保持。把不同目标的优化混成一个总速度数字，会掩盖功能代价和测量边界，因此报告分别保留吞吐、时序、面积和正确性证据。

P4 的失败说明局部算术变快不足以完成集成资格。它在原有门槛中引入更多失败项，应按失败类型追溯组合路径、协议状态和父级共享资源，而不能只挑选局部速度较好的样本替换基线。最终采用 0f 是基于当前完整筛选结果的工程选择；未来若修订 P4，需要作为新的版本重新执行同一验证合同。这样既保留可优化的空间，也防止未通过门槛的候选无意进入默认交付。

第二阶段交付的核心不是若干通过次数，而是一份可追溯的设计闭环：数学定义对应位宽，指令合同对应译码和配置依赖，事务身份对应弹性管线及取消，父级资源对应门控和共享输出，软件可见结果对应写回及提交。后续阶段可以沿此闭环建立独立模型和性能实验；若其中任一环的输入、观察点或版本发生变化，应先更新合同，再解释新旧结果的可比性。

2.7.16 回归矩阵中的组合关系

测试项数量应服务于失效机理，而不是追求一个孤立的总数。算术极值与随机输入解决数值正确性，配置交错解决依赖绑定，别名解决源快照与目的分配，反压解决有效位和载荷稳定，取消解决年龄与复位优先级，父级混合操作解决共享资源。一个测试可以覆盖多个维度，但记录应指出其主断言及额外观察点。这样，当某一项失败时，能够快速找到最相关的输入与波形；当实现变化时，也能判断哪些测试必须重跑。

为了避免组合爆炸，可先对每一维构造最小可区分情境，再挑选交叉点。比如正常单事务可检查基础算术和响应格式；双事务且不同目的寄存器可检查连续接受；双事务加反压可检查载荷稳定；环回 ROB 加选择性取消可检查年龄语义；取消后首笔 DIV 和点积交替可检查父级释放。并不是所有输入值都要与所有控制状态笛卡尔相乘，但每个高风险交叉都应能在矩阵中定位到明确的用例和预期。

随机测试也需要确定的覆盖边界。种子和生成规则决定能否重放，输入约束决定是否覆盖负数、极值、零、高位哨兵和寄存器别名。若随机生成器默认均匀抽取完整向量，特定十八位溢出端点几乎不可能自然出现，因此定向极值不能由万组随机样本替代。反过来，定向样本也难以代表一般输入组合。报告同时保留两类证据，并分别说明它们为何存在。

2.7.17 从日志到源码的审查步骤

审查设计时，先以数学参考值和事务身份写出预期，再阅读源码中的状态更新条件。对于每个槽位，列出被接受、被推进、被输出、被取消和被复位五种转移的有效位变化，并检查同拍组合。随后在仿真波形中选择至少一条对应轨迹，比较边沿前后状态。只看到最终通过摘要不足以确认源码条件正确，因为测试可能未制造出最危险的同拍组合。

源码评审还要关注“未发生动作”的周期。反压时输出载荷不能漂移，空闲时不应出现有效响应，取消后被清除的身份不能再次出现，时钟门控下未接受的请求不能被错误记账。这些不变量在正常完成日志中可能没有显式文本，因此需要检查断言或观察器是否确实在每个相关周期运行。若检查器只在 ready 为一时观察输出，就无法证明 valid 等待期间的稳定性。

对失败项应保留最短的复现输入、预期与实际差异、关联源码版本和修复后重跑范围。若修复发生在父级输出选择，局部算术通过不能代替父级混合操作重跑；若修复发生在乘积位宽，控制协议检查仍需至少确认没有被状态重构连带影响。回归范围依据修改影响确定，既不机械地仅跑一个用例，也不把所有历史日志不加区分地重新解释为新版本证据。

2.7.18 时序与吞吐取舍的解释

弹性槽提供的是输入与输出之间的缓冲容量，它改善持续请求或短暂下游阻塞时的利用率，却引入额外的状态寄存器和 ready 路径。对独立指令流，流水重叠可以降低平均每条指令的间隔；对严格依赖链，下一条必须等前一条写回，缓冲容量不一定产生同样收益。报告中的性能判断需要与这两类工作负载分别配对，不能只从微架构结构图推出实际加速比。

综合成本也有层级。乘法树位宽、P/R 寄存器规模、控制比较器以及父级选择逻辑分别贡献不同路径和面积。H1 收窄结果状态可能减少寄存器位数，但若增加末端拼接或改变布线，最差时序端点可能转移。P4 缩短内部算术拍数也可能使组合路径穿越更多逻辑，导致父级 ready 或门控使能的余量变差。优化审查需要报告路径端点和约束，而不能把单一面积或延迟变化当作全局结论。

2.7.19 面向下一阶段的接口冻结

第二阶段结束时需要交给第三阶段的是可独立实现的架构语义，而非 RTL 内部的乘法和槽位细节。参考模型应得到指令字、源向量、控制状态和异常条件，并返回预期的目的向量及异常结果。它不应依赖 P/R 具体拍数，否则任何合法的流水优化都会迫使参考模型变化。相反，硬件性能实验需要清楚记录当前实现版本和开关，因为同一架构语义可以对应不同的周期和面积。

软件测试的输入镜像、编译器汇编路径和配置前置条件也应冻结。若第三阶段换用普通 C 接口或新的链接布局，应先核对最终机器指令仍为目标编码，再说明与第二阶段

裸指令测试的共同部分和新增部分。这样，差分验证能够沿用数学与异常合同，性能测量又能诚实反映软件包装的额外开销。两阶段的结果在同一指令语义上衔接，但不会因为软件入口相似就被误认为同一测量合同。

2.7.20 回归失败的归属与收口

回归出现失败时，首先核对失败对应的构建摘要、优化开关和输入集合。若源码未变而检查器改变，应重新判断先前通过记录的有效范围；若源码改变但沿用了旧仿真产物，则本次失败或通过都不能直接归属给新版本。定位时从第一个可复现的错误观察点出发，追溯输入接受、槽位迁移、响应选择和目的写回，尽量保留最短事务序列。后续修订应在同一序列上先证明错误消失，再运行该修订影响到的完整矩阵。

对于多个同时失败的断言，应区分初始错误与连锁症状。例如错误的 ready 可能引起输入重复接受，继而出现结果错位、身份错位和最终计数不守恒；只修补末端比较器不会恢复协议。相反，单个高位结果错误若不影响握手和身份，就应优先检查结果拼接与参考模型的完整向量比较。按因果链归属失败，可以减少针对表面症状的补丁，并使新增测试覆盖真正的触发条件。

收口记录至少包括修订前失败轨迹、源码改动、修订后同输入结果和受影响层级的回归结果。尚未得到父级或全核证据的修订，只能标注为局部修复。若优化开关在正常交付中仍关闭，应在构建配置和报告结论中同时写明，防止读者把候选实验的通过误认为默认实现的通过。这个版本纪律是第三阶段开展配对性能测试和软件集成的基础。

2.7.21 写回身份与功能结果的联合检查

向量点积的数学参考值可以由两个源向量独立计算，但它不能告诉检查器该值是否写入了正确目的。测试平台因此为每个接受事务绑定一个身份记录，包含目的物理寄存器、ROB 位置、指令控制及期望结果。响应到达时先核对身份，再核对完整一百二十八位数据。若先按结果值寻找匹配事务，两笔恰好得到同一结果的请求可能被错误互换而不被发现。零结果、相同源向量和不同目的寄存器的交错请求尤其适合暴露这一盲点。

写回使能与异常字段同样需要随身份核查。异常指令可以在执行路径中占有状态，但不能以正常结果覆盖架构目的；被取消的指令在局部槽位里保留旧数据，也不能因 ready 恢复而重新输出。检查器在每次接受、响应和取消时更新账本，并在结束时核对所有身份的去向。该账本提供了比单纯比较输出顺序更强的约束，也能分辨协议漏响应与结果错误。

完整路径复验还应覆盖目的别名和连续配置变化。若两条指令的源与目的存在交叉别名，重命名阶段必须固定各自的源版本；若配置指令夹在两条点积之间，执行端必须持有各自对应的 VL 与 vtype 快照。最终软件结果可能在少数输入上偶然相同，因而需要结合调度记录、响应身份和异常状态判断。只有这些字段与数学结果共同成立，才可将局部算术通过提升为全核写回路径通过。

2.8 内部 ISA 合同、数学边界和测试向量

本节整理固定项目材料中的详细方法和原始合同。设计阶段的“拟实现”“必须验证”等表述属于原合同约束，当前完成状态以本报告正文和 R5/P5 证据为准。原日期、比较合同及未测边界按来源保留。

来源：team/spec/vdot.md

2.8.1 数学语义与目的位布局

2.8.1.1 源元素编号

从指令执行前的源寄存器状态读取：

```
byte_a[i] = vs1_before[8*i+7 : 8*i]
byte_b[i] = vs2_before[8*i+7 : 8*i]      i = 0…7
signed8(x) = x            if x < 128
           x - 256      otherwise
```

源 [127:64] 不参与运算。内存加载测试使用既有小端裸机环境，数组的第 0 个字节对应源 [7:0]；不能把打印的高位十六进制组误认成 lane 0。

2.8.1.2 精确范围与位宽

单项乘积范围为 [-16256, 16384]，因此八项精确和为：

```
S_min = 8 × (-128) × 127    = -130048
S_max = 8 × (-128) × (-128) =  131072
```

有符号 18 位最大值为 131071，不能覆盖 S_max；至少需要 **19 位有符号数**。原型使用 32 位结果，合法输入不发生 32 位溢出。

RTL 可先采用 8 个 signed 8×8 乘积与加法树：乘积 16 位，按二项、四项、八项归并时分别至少用 17、18、19 位，再符号扩展到 32 位。也可全部以 32 位求和。A 决定结构和流水级，但所有中间结果必须保留符号和足够位宽；不得先用窄位宽加法溢出，再扩展结果。

2.8.1.3 输出定义

```
127                          32 31                  0
+-----------------------------------+--------------------------+
|          全部为 0                 |    S 的 32 位补码        |
+-----------------------------------+--------------------------+
```

示例：S=-8 时，完整目的值为 0x00000000_00000000_00000000_FFFFFFF8。负数只在低 32 位内采用补码，高 96 位仍为零。本文中的“结果字段”固定为 32 位，不把它称为 SEW=8 下的普通第 0 个元素；否则容易错误截断为 0xF8。

这是自定义指令的整寄存器写回约定。vl=8 用来限制合法输入配置，不控制目的只写低 8 个字节；不得由通用 tail/mask 合并逻辑保留旧目的高位，或把高位改成全 1。

2.8.1.4 参考伪代码

以下描述架构效果，不指定时钟周期和具体 NEMU 接口：

```
def execute_vdot(inst, state):
  # 必须先完成编码、向量状态和配置检查；失败则走非法指令路径。
  require_vdot_encoding_and_supported_state(inst, state)

  src1 = read_full_vector_before_write(inst.vs1)  # 128-bit snapshot
  src2 = read_full_vector_before_write(inst.vs2)  # 128-bit snapshot
  total = 0                              # 数学整数，不模拟 RTL 截断
  for i in range(8):
    a = (src1 >> (8 * i)) & 0xff
    b = (src2 >> (8 * i)) & 0xff
    a = a if a < 128 else a - 256
    b = b if b < 128 else b - 256
    total += a * b

  result128 = total & 0xffffffff             # 仅编码低 32 位补码
  write_full_vector(inst.vd, result128)       # 高 96 位因此为零
  # vl/vtype/vxrm/vxsat/vcsr 不变；vstart 仍为 0；向量状态按写回置脏。
```

独立数学模型只计算输入数据与输出；完整指令模型另外实现编码、配置检查和异常，不能把 Python assert 当成处理器异常机制。

2.8.2 合法配置、别名、异常与状态

2.8.2.1 配置合同

表 2-5 配置合同

条件	原型行为及测试要求
VLEN	目标固定为 128；RTL 构建与 NEMU 配置均检查该常量。不承诺跨 VLEN 可移植性
向量状态	运行环境支持并启用 V；mstatus.VS 不能为 Off；沿用处理器已有特权态检查
vtype	vill=0，vsew=000（e8），vlmul=000（m1）；其他非法/保留 vtype 不接受
vl	必须等于 8。vl=0、1、7、9、16 等域外值均不执行点积
vstart	必须为 0；非零值不表示跳过前几个乘积，也不被静默清零后继续执行

表 2-5 配置合同（续）

条件	原型行为及测试要求
vm	必须为 1；vm=0 的相邻编码不属于本版合法指令
vta/vma	tu/mu、tu/ma、ta/mu、ta/ma 均合法；输出仍精确遵循第 3.3 节

推荐首个测试程序使用 vsetivli zero, 8, e8, m1, tu, mu，读取并核对实际 vl/vtype，并确保 vstart=0。该配置指令是准备工作，本身不能让未扩展的处理器识别 vdot。

这是有意收窄的 custom 指令原型域。域外统一非法是团队选择；尤其 vl=0 在本版不作为空操作处理。标准 RVV 指令的通用循环、tail 和 restart 规则不能未经核对直接套入。有关标准向量状态、策略位和重启机制的背景参考 RISC-V Vector 1.0 文档；它没有定义本项目的 vdot 行为。

2.8.2.2 寄存器别名

vd/vs1/vs2 均为 v0…v31，每个操作数只占一个寄存器，无偶数寄存器对齐要求。v0 在本版 vm=1 下可作为普通源或目的，不是零寄存器。

所有源值以指令前状态为准，再写目的。NEMU 应先缓存所需源数据，不能先清空 vd 再逐项读取源。RTL 应正确维护源、目的重命名关系，不能把目的新物理寄存器当作输入。由于完整覆盖目的，算术上不需要旧 vd；物理接入是否存在通用旧目的端口由 A 审查，不得让它改变结果。

2.8.2.3 成功执行与失败执行

成功执行时：

- 只更新一个向量目的寄存器的完整 128 位；其余向量寄存器、标量寄存器、浮点寄存器、内存不变。
- vl、vtype、vlenb、vxrm、vxsat、vcsr 不变；合法初态下 vstart 仍为 0，不能把原有 vxsat=1 清除。
- 沿用现有向量状态置脏机制；不产生浮点异常标志，不访问数据存储器。
- 正常进入下一条指令（本编码长度 4 字节），退休与计数行为沿用处理器正常指令机制。

匹配本指令但配置不合法时，**原型目标行为**为精确非法指令异常（异常码 2）。目的寄存器、运算相关向量 CSR 和内存不产生本指令的部分更新；陷阱 PC 指向该指令。陷阱入口对特权 CSR 的正常更新不属于“不变”范围，tval 填写和异常优先级沿用已验证

的现有机制。测试在状态可达时优先使用单一非法条件，不在本版另造一套异常优先级；存在状态联动时按第 7.2 节记录。

VLEN 不符优先在构建/加载配置时拒绝；若未来允许在其他 VLEN 机器执行相同编码，应另立版本规定其行为。

乱序执行中的请求若被 redirect/flush 取消，不得形成已提交的目的更新。首版不提供从中间 lane 恢复的架构状态；中断与取消复用现有精确提交机制。这些均需全核验证后才能勾选完成。

2.8.3 候选机器编码

2.8.3.1 编码选择

【原型约定】选择 32 位 custom-0 主 opcode 0x0b。官方 opcode 表为自定义扩展预留 custom 空间；这不表示下列 funct 字段由官方分配。标准 opcode 表

表 2-6 编码选择

位段	字段	v0.1 候选值
[31:26]	funct6	000000
[25]	vm	1；本版只支持无掩码
[24:20]	vs2	第二个编码源寄存器编号，0～31
[19:15]	vs1	第一个编码源寄存器编号，0～31
[14:12]	funct3	000
[11:7]	vd	目的向量寄存器编号，0～31
[6:0]	opcode	0001011（0x0b）

汇编记法沿用向量常用顺序 vdot.vv vd, vs2, vs1。数学公式中 vs1/vs2 的名称与位段一致；点积交换律会掩盖源颠倒，所以编码测试还必须检查字段提取本身。

```
MATCH_VDOT = 0x0200000b
MASK_VDOT  = 0xfe00707f
识别条件： (instruction & MASK_VDOT) == MATCH_VDOT
编码公式： MATCH_VDOT | (vs2 << 20) | (vs1 << 15) | (vd << 7)
BitPat：  b0000001??????????000?????0001011
```

例：vdot.vv v10, v8, v9 的候选机器字为 0x0284850b；小端指令字节为 0b 85 84 02。寄存器编号为向量寄存器编号，即使字段位置与 R-type 相似，也不能在主译码中把它们当作通用整数寄存器。

vm=0 或 funct 字段不符时不匹配本版 vdot；若未被其他合法扩展占用，应走默认非法指令路径。不得通过缩小 mask 接受未定义的相邻编码。

2.8.3.2 工具链与冲突检查边界

首版先用独立.S 文件中的.word 0x0284850b 或经过校验的编码宏生成上述机器字。后续封装由 B 负责；宏必须检查寄存器编号范围。未新增助记符支持前，反汇编可能显示原始字或未知指令，应核验机器字和字段，而不能以“打印出了 vdot 名称”作为前置条件。

如使用 C 内联汇编，须另行处理寄存器绑定、clobber 与编译器重排；不要把.word 放进没有约束的内联汇编后假定编译器知道它读写向量寄存器。

2026-10-11 的只读静态检查在锁定版本的 **XiangShan 当前实际译码表和 NEMU 主译码表**中未发现与上述候选 mask/match 的交叠。已有 nemu_trap/TRAP 使用 0x6b，并非 0x0b。rocket-chip 依赖中确有与候选重叠的 CUSTOM0 通配定义，但属于 Rocket RoCC 译码体系，未接入本版香山实际译码表；未来若启用该通路，必须重新审查分配与仲裁。此结论只覆盖已检查的静态模式，不能代替接入后的译码单测与全核回归；候选编码尚未实现。

2.8.4 RTL、NEMU 与验证共同遵守的接入要求

2.8.4.1 锁定版本的阅读入口

表 2-7 锁定版本的阅读入口

子系统	相对路径	需要核查和完成的事情
XiangShan 指令模式	src/main/scala/xiangshan/backend/decode/Instructions.scala	增加候选模式，检查邻近编码和与既有扩展的冲突
XiangShan 向量分类	src/main/scala/xiangshan/backend/decode/isa/bitfield/RiscvInst.scala	isVecArith 当前只识别 OP-V；custom-0 不会自动被识别为向量算术
XiangShan 译码控制	src/main/scala/xiangshan/backend/decode/DecodeUnit.scala	正确的向量源/目的类型、配置依赖、合法性检查、微操作和写回控制
XiangShan 执行接入	src/main/scala/xiangshan/backend/fu/FuConfig.scala，fu/wrapper/，fu/vector/	选择可复用通路或新单元、执行资源、结果位宽、完整目的写回和取消机制
NEMU 主译码	src/isa/riscv64/instr/decode.c	分派 custom 主 opcode；检查向量预分类和执行入口，不能只加 RVV 子表

表 2-7 锁定版本的阅读入口（续）

子系统	相对路径	需要核查和完成的事情
NEMU 向量实现	src/isa/riscv64/instr/rvv/decode.h、exec.h、vcompute.h、vcompute_impl.c	编码、配置检查与独立算术实现；避免误复用通用 tail/restart 行为
NEMU 寄存器访问	src/isa/riscv64/instr/rvv/vreg.h、vreg_impl.c	有符号源读取、先读后写、完整 128 位写入
NEMU 声明与差分状态	src/isa/riscv64/include/isa-all-instr.h、src/isa/riscv64/difftest/ref.c	执行入口声明、向量和 CSR 状态同步，确认实际比较范围

这些是源码阅读入口，不要求机械修改表中的每一个文件。来源：固定 XiangShan 译码目录、固定 NEMU 主译码、固定 NEMU RVV 目录。

以下是本版源码核查发现的具体接入风险；行号对应表中固定硬件/NEMU 提交：

- RiscvInst.scala:93 的向量分类需要识别精确 vdot 模式，不能把全部 custom-0 都归为向量。DecodeUnit.scala:897 的向量禁用检查还依赖 FU 分类，因此只加 BitPat 不够。
- DecodeUnit.scala:775 的输入带 vtype/vstart，未直接带实际 vl；vl=8 限制须通过正确的 VL 依赖，在能取得 srcVConfig.vl 的路径检查，例如 fu/vector/VecPipedFuncUnit.scala:54，并接通非法指令异常输出。
- DecodeStage.scala:236 对 v0 目的有单独写使能，重命名也有相应通路。允许 v0 就必须覆盖它，不能只测 v8/v9/v10 后认定全部寄存器均可用。
- NEMU 的 instr/rvv/decode.h:167 在既有 vopivv 子表中把 funct6=0 解释为 vadd，且不再检查主 opcode。**不能把 custom-0 直接分派到现有 OP_V 执行子表**；应建立独立 vdot 执行分派，可以复用字段解析和寄存器访问。状态检查/置脏参考 instr/vp.c 的 vp_enable()、vp_set_dirty()。

2.8.4.2 硬件接口边界

A 在 T06 维护 RTL 接口与接入设计（2026-10-12 已形成第一版草案），B/C 复核具体端口与时序，人工评审待记录。本文只锁定必须实现的性质，不虚构固定周期数：

1. 运算请求携带正确的两个向量源值和请求身份；配置检查必须使用与该指令对应的向量状态。

2. 只有请求真正被接受后才进入执行；输出反压时保持结果及身份，不能丢失、重复响应。

3. 正常完成形成一个完整 128 位架构目的结果；内部微操作数和流水级由接入设计决定。

4. 结果与目的物理寄存器、ROB/取消标识保持一致；flush 后不能提交已取消的结果。

5. 全寄存器写回绕开或正确配置通用 tail/mask 合并逻辑，保留高 96 位清零要求。

6. 功能单元延迟和发射间隔分别定义、分别测量；本文不预先承诺“一周期”或加速倍数。

2.8.4.3 NEMU 与交付约束

B 先写独立数学模型，再实现 NEMU。不要把 RTL 的加法树、中间截断或共享错误复制成参考模型；不能通过跳过 vdot 的 DiffTest 比较获得“通过”。C 应注入一次可控错误，证明比较路径能发现目的值或高位清零错误。

共享参考库与 standalone NEMU 应分开构建，保存各自提交、配置和实际加载路径，避免覆盖已有环境产物。NEMU 修改应提交至团队可访问的 NEMU 仓库或采用明确可复现的补丁交付；确定方式并更新版本引用后，其他成员才算能够完整复现。

2.8.5 测试与验收合同

本节是**待实现的测试清单**。表中期望值由数学定义得出，不是已完成的仿真结果。所有功能测试必须比较完整 128 位目的值。

2.8.5.1 定向算术向量

[x]×8 表示重复 8 次。列表按 lane 0→7 排列，未另行说明时源高 64 位为零；输出高 96 位在所有用例中均为零。

表 2-8 定向算术向量

ID	源 A 的低 8 项	源 B 的低 8 项	S（十进制）	目的低 32 位（十六进制）
D00	[0]×8	[127]×8	0	00000000
D01	[1]×8	[1]×8	8	00000008
D02	[1,2,3,4,5,6,7,8]	同 A	204	000000CC
D03	[-1]×8	[1]×8	-8	FFFFFFF8
D04	[-1]×8	[-1]×8	8	00000008
D05	[127]×8	[127]×8	129032	0001F808
D06	[-128]×8	[127]×8	-130048	FFFE0400

表 2-8 定向算术向量（续）

ID	源 A 的低 8 项	源 B 的低 8 项	S（十进制）	目的低 32 位（十六进制）
D07	[-128]×8	[-128]×8	131072	00020000
D08	[1,-2,3,-4,5,-6,7,-8]	[8,7,6,5,4,3,2,1]	0	00000000
D09	[-128,0,0,0,0,0,0,0]	[-1,0,0,0,0,0,0,0]	128	00000080
D10	[0,0,0,0,0,0,0,-128]	[0,0,0,0,0,0,0,127]	-16256	FFFFC080

2.8.5.2 数据布局、状态与控制

表 2-9 数据布局、状态与控制

ID	测试内容	判定标准
L01	逐 lane 一热：只令 A 的 lane i 非零，B 的 8 项互不相同，i 遍历 0～7	每次等于对应那一项乘积，检测遗漏和单侧错位
L02	保持低 8 项不变，两个源高 64 位分别换成随机值、全 FF、交替 55/AA	结果完全相同
L03	独立目的寄存器预填全 FF，再执行 D03	目的完整值为 00000000_00000000_00000000_FFFFFFF8
L04	不重叠、vd=vs1、vd=vs2、vs1=vs2、三者相同；含 v0/v31	使用实际执行前源快照算期望；别名不能使源被提前破坏
L05	四种 vta/vma 组合	每种均得到相同完整目的值，保留原配置
S01	vxrm 四种值、vxsat 0/1 组合，预置后执行合法指令	算术结果不变；上述状态及其 vcsr 映射保留
S02	检查 vl/vtype/vstart、非目的寄存器与内存	除正常提交、向量置脏等既有机制外，无额外副作用
E01	逐项改变 SEW、LMUL、vl、vstart、vill，或关闭向量状态	模型显式拒绝；完整指令实现发生非法指令异常，无部分结果

表 2-9 数据布局、状态与控制（续）

ID	测试内容	判定标准
E02	vm=0、逐个翻转固定 funct 位；合法寄存器字段遍历	只识别精确 mask/match；确认相邻模式未被误译码为 vdot
C01	连续请求、输出反压、结果依赖、同一目的连续写、flush	无丢失/重复/错序提交，取消后无已提交写回
R01	原 hello/vadd 及相关向量回归	保持正确；vdot 新路径不破坏旧指令

L01 无法检测“两侧同时采用相同排列”的内部置换，因为点积对这种置换本来就不变；不要为无法观察且不影响语义的内部排列编造失败标准。CSR/非法配置测试应使用合法准备指令并保存实际 CSR 值，避免测试准备过程就已失败。

非法配置须通过实际可达状态构造：用标准 vset* 指令设置非法 vtype 时，vill=1 与 vl=0 联动，此例只验证目标 vdot 的非法异常及无部分更新，不能据此声称单独覆盖了 vill 检查或异常优先级。测试非零 vstart 时，应在向量配置、加载等准备完成后最后写入，避免被准备指令清零。向量状态关闭案例也应先完成向量数据准备，最后再关闭 VS。

2.8.5.3 随机验证与证据

- 软件模型/算术单元起步使用固定种子 0x20261011，至少 10,000 组 signed int8 输入；全核冒烟先用上述定向用例，再选 32 组固定随机用例。数量是初始工程安排，不是官方评分门槛。
- 保存生成器版本、实际输入、完整期望值、合同版本。仅有随机种子不一定能跨语言复现随机数序列。
- 交叉检查交换两个源、同时重排两边低 8 项等不变量。涉及 int8 取负时排除 -128，避免把不可表示的 +128 当成合法输入。
- 失败应输出指令字、PC、配置、源/目的编号、指令前源值、完整实际/期望目的值；定向用例输出用例 ID。
- 独立模型、RTL 单元、NEMU、全核 DiffTest 分别保存结果，不能用某一层通过替代其他层。

软件读取结果时要覆盖全部 16 字节。例如执行后切换到 e32,m1,vl=4，保存四个 32 位元素，再恢复约定的配置；C 应区分“被测指令副作用”和“测试程序为读回结果而改变的配置”。只保存 e8 的 lane 0 会漏掉错误。

2.8.6 RVV 性能基线必须保持的等价性

【题目明确】需要用现有 RVV 指令组合完成相同功能，并比较延迟。候选基线按“8 位有符号输入 → 16 位精确乘积 → 32 位扩宽归约”设计，例如研究 vwmul.vv 与 vwredsum.vs 的组合；中间切换的 SEW/LMUL、寄存器分组和零初值必须匹配。这里不是已通过验证的汇编实现。

B/C 验收基线时必须逐项确认：

1. 只计算源低 8 项，从零开始，不把旧目的或源高半部分加入。

2. 不使用会在 8/16 位提前截断的组合；D06、D07 必须通过。

3. 最终目的低 32 位及高 96 位与第 3 节完全相同；普通归约指令的尾部策略不能代替显式验证。

4. 双方按共同约定的可观察入口/出口状态验证功能等价：目的 128 位结果及向量 CSR 必须一致，vl/vtype 恢复实际入态。允许破坏的临时（scratch）寄存器必须预先列明并排除于比较集合，源/目的别名用例另行证明等价。该口径不能被称为“除目的外所有架构状态都保持不变”；第 4 节的严格副作用限制仍适用于单条 vdot 本身。

5. 分别报告核心算术和上述合同范围内完整序列的周期；必要的清零、配置切换、恢复和搬运开销不得只从基线一侧隐藏。装载、存储是否计时，两边保持一致。

6. 同一硬件基线、同一输入和测量边界下比较；声明测量的是依赖延迟还是吞吐。Verilator 主机运行秒数不能当作处理器周期加速比。

具体序列与测量事件由 B/C 在后续 team/spec/measurement.md（待创建）中确定。本文不承诺加速比或获奖分数，先以完整性、正确性和可复现证据为验收顺序，再做流水线和扩展优化。

2.8.7 歧义清单与变更流程

下列清单供 B 整理后，通过比赛已有咨询渠道向命题方确认。**截至本版没有在仓库归档答复，也没有代表团队发出外部消息。**

表 2-10 歧义清单与变更流程

编号/优先级	需要确认的问题	本版候选及受影响内容
Q01 / P0	图中的 8 项是否为最终验收语义？是否明确取低编号 8 项？	固定低 8 项；影响读取范围、算术规模、测试和基线
Q02 / P0	“vd 第一个元素”的位宽是什么？是否接受低 32 位精确结果、高 96 位清零？	第 3.3 节；影响目的格式、写回和完整结果比较

表 2-10 歧义清单与变更流程（续）

编号/优先级	需要确认的问题	本版候选及受影响内容
Q03 / P0	是否要求精确结果？是否另有截断、饱和或旧 vd 累加规则？	精确从零求和；影响数据通路与边界结果
Q04 / P0	vl/LMUL/mask/vstart 的必测范围是什么？是否接受第 4 节的受限原型域？	固定 e8/m1/vl8/vstart0、无掩码；影响控制及异常范围
Q05 / P1	vta/vma 是否影响“其他通道清零”？是否要求源目的重叠支持？	策略位不改变完整结果，全部单寄存器别名支持
Q06 / P1	是否有指定或禁止的指令编码？是否允许使用 custom-0？	第 5 节；影响 RTL、NEMU 与汇编封装同步
Q07 / P1	延迟比较是否有指定输入/出口状态、计时边界或基线要求？	按第 8 节先验证等价，再分别报告各项开销

如获得回复，在 team/spec/clarifications/（待创建）保存日期、来源和原文/截图，再在表中补充结果。没有答复不等于官方认可。等待期间可按明确标记的原型版本开展模型、算术单元和用例工作。

行为变更流程：B 提出“旧行为→新行为、依据、受影响用例”；A/C 评审；升级合同标识；同一轮更新 RTL、NEMU、编码封装和测试数据。旧版本结果仍保留版本标签，不能混入新版性能统计。

2.9 两槽弹性状态、父级协议及验证不变量

本节整理固定项目材料中的详细方法和原始合同。设计阶段的“拟实现”“必须验证”等表述属于原合同约束，当前完成状态以本报告正文和 R5/P5 证据为准。原日期、比较合同及未测边界按来源保留。

来源：team/optimizations/o2/CONTRACT.md

2.9.1 接口与状态

方案 A 保留 FuConfig.piped=false、UncertainLatency()；这里 piped 字段属于现有框架配置约束，不表示新模块内部不能有两个流水槽。该字段若为 true，现有 FuConfig 要求 certain latency，不能为名称好看而更改时序合同。

新候选使用独立的 FuncUnit 实现（可复用 VecFuncUnitAlias 读取输入），不得直接依赖 VecNonPipedFuncUnit 的一份 RegEnable/DataHoldBypass 输出身份；也不得套用要求 certain latency 的 VecPipedFuncUnit。

表 2-11　接口与状态

槽位	有效状态与数据	必须一同保存的身份
P：乘积槽	valid、8 个 16 位有符号乘积、输入合法性	接受时的 ROB、pdest、vecWen/v0Wen、需要输出的完整 vpu 控制及 perfDebugInfo；字段清单按 FuncUnitOutput 逐项核对
R：结果槽	valid、完整 128 位结果、异常向量	从 P 原样转移的同一包身份，不能从当前输入取得

非法请求照常占一个槽并终结一次，不能在算术或元数据通路上提前丢弃。v0 与普通向量目的互相交替时，各自写使能及 pdest 不得串线。所有输出控制、异常及 perf 信息由 R 提供；输入在反压期间任意变化也不能改变旧响应。

给父级提供明确的功能状态端口：两个实际 valid 或其精确 occupancy/empty 归约。时钟门控和派发使用这些功能状态，不能用 printf、调试 seq、tap 日志或推测的最近一条 ROB 替代。若父级为验证而保存副本计数，必须每拍断言与 FU 真实状态一致。

2.9.2　每拍转移定义

以下量在同一时钟边沿前计算，ready/valid 遵循 Decoupled 握手。P、R 各自调用该请求的 robIdx.needFlush(io.flush)，保留硬件 RobPtr 的 flag 与环绕语义。

killP = P.valid && P.rob.needFlush(flush)

killR = R.valid && R.rob.needFlush(flush)

liveP = P.valid && !killP && !reset

liveR = R.valid && !killR && !reset

out.valid = liveR

out.fire = out.valid && out.ready

rReady = !liveR || out.ready

move = liveP && rReady

pReady = !liveP || rReady

in.ready = pReady && parentAllowsVDot && !flush.valid && !reset

in.fire = in.valid && in.ready

nextR.valid = (liveR && !out.fire) || move

nextP.valid = (liveP && !move) || in.fire

当 move 为真，R 锁存 P 的归约结果和整包身份；当 in.fire 为真，P 锁存新输入的乘积和整包身份。move 使用边沿前的旧 P，绝不使用同拍新输入。reset 强制两个 valid 及父级占用清零；无效槽的数据无需清零，但不得影响可见输出。

表 2-12　每拍转移定义

边沿	接受	槽间转移	输出握手
E0	A 进入 P	无	无
E1	B 进入 P	A 进入 R	无
E2	C 进入 P	B 进入 R	A
E3	D 进入 P	C 进入 R	B

表中为从空管线开始、持续 ready 且无取消的目标时序，不是测得的 O2 波形。已有 R 但输出阻塞时，空 P 仍可接收一条；两槽满且 R 阻塞时拒绝输入。R 被取消而 P 幸存时允许 P 进入腾出的 R；P 被取消而 R 幸存时不能误清 R。

重定向当拍统一不接受新请求；未被取消的旧 R 可正常握手，幸存 P 可转移。被取消请求的 KILL 优先于它的 RESPONSE，二者不能同时计数。无关重定向不能破坏已有停顿响应。RESET 优先级最高，复位当拍无输入/输出握手。计数时使用 effectiveKillP=killP&&!reset、effectiveKillR=killR&&!reset；RESET 丢弃数取边沿前两个 valid 之和，reset 与 flush 同拍不能把同一请求同时记为取消和复位丢弃。

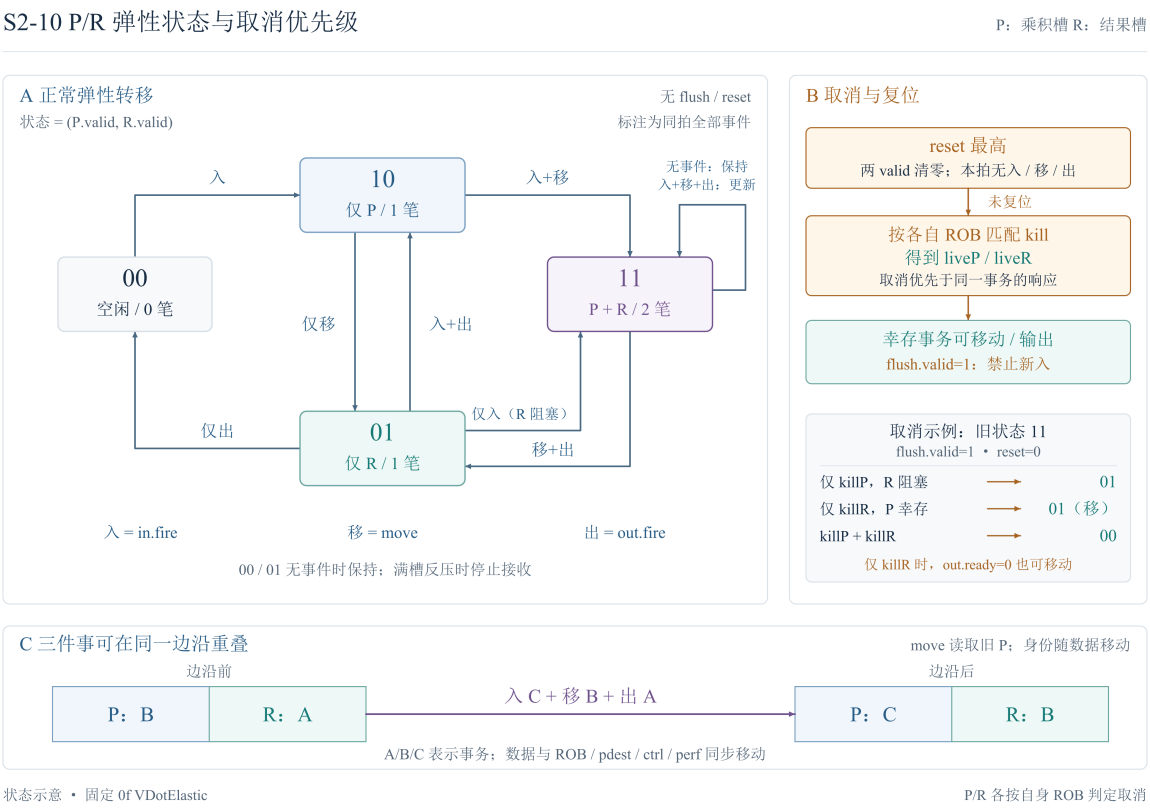


图 2-10　P/R 弹性状态与取消优先级

注：状态位按 (P 有效,R 有效) 排列，区分输入接收、P 至 R 移动、输出握手和保持；reset 及 ROB 取消优先处理，事务身份随数据移动。

来源与范围：用户提供示意图；依据：本文状态方程、0f 实现及 O2 合同不变量。版本范围：历史合同仅作依据，图中所有行为以最终 0f 核对。

2.9.3　父级资源、派发与除法切换

当前 VFEX4 含 VDot、VIDiv、VFDivSqrt；共享输出是 one-hot 选择和 ready 广播，缺少多 FU 同时完成仲裁。本候选维持互斥：

1. 有任意 vdot 在途时，只能向 vdot 追加请求；DIV 禁止接收。

2. 任意 DIV 在途时，vdot 禁止接收，保持已有 DIV 串行行为。

3. vdot 最后响应或取消当拍不切换 DIV；在下一拍确认两个槽确实空、无旧 vdot 输出后，才可接受 DIV。

4. DIV 完成/取消后也必须确认旧输出已经撤销、子单元具备合法接收状态，再放行 vdot；不依赖取消当拍旧 DIV valid 立即变低的假设。

5. 所有跨 FU 切换都不得出现两个 out.valid 同时有效。DIV 自身已有取消当拍行为需原样观察并按现有全核合同解释；不能为了让新检查器通过而改写旧预期。

6. 持续 vdot 到来可能使排空切换延迟；不承诺新的公平调度。混合负载测试须记录等待，有限输入停止后必须能排空并完成 DIV。

在 O2 专用的 VFEX4 路径局部选择被选 FU 的 ready，或采用等价的原子派发实

现；不修改所有 EXU 共享的 Dispatcher 语义。现有 Dispatcher 把所有 ready 相与，但 child.valid 并未与总 ready 相与，不能仅放宽父级 busy 而让 child 单独接受。必须断言：

parent.in.fire == OR(all child.in.fire)

PopCount(all child.in.fire) <= 1

child.in.fire -> parent.in.fire && selected(child)

父级不能继续用单个 busy 和最近输入 RobPtr 表示多个 vdot。vdot 占用从真实槽状态取得；DIV 占用及身份独立保存。任何一笔 vdot 响应/取消只释放该事务，不能把另一个在途请求一起“释放”。

输入选择只依赖已注册占用、当前合法选择及下游 ready；不得构成 valid/ready 组合环。选择后未握手的输入仍由上游保存，FU 不得提前更新身份。

2.9.4 门控、唤醒与写回

- 当前不定时延门控只用一个 uncer_en_reg，out.fire 且没有 in.fire 时会清零；O2 仍有 P 请求时这会错误关钟。vdot 专用使能必须覆盖当前任一槽有效、当前输入接受以及必要复位/清除边沿。
- 当前槽有效时，即使 flush 将其全部取消，也必须保留处理本次清除所需的时钟边沿；排空后再关门。父级状态逻辑位于未门控时钟域。
- 只对 O2 vdot 改门控;DIV 及其他 EXU 保持现状。真实父级测试使用真实 ClockGate，不能用常开时钟替代最终验证。必须明确 reset 在实际门控使能/复位传播中的接法，并覆盖空槽且已关钟时 reset、非空时 reset、cancel/reset 同拍及随后重新唤醒；抽象状态公式不能证明复位脉冲确实到达门控后的寄存器。
- A 保留既有不定时延完成/唤醒与实际写回仲裁 ready。某次输出未被接受时，R 必须保持全包稳定；不得假定写回总是 ready。
- 不新增固定延迟提前唤醒、写回预留或旁路承诺。若发现现有 issue 反馈限制 II=1，先记录并评估，不自动扩大到调度重构。
- 若转选 B，必须另修合同并重审 FuConfig、ctrlPipe、调度/唤醒、寄存器读端口及写回预留；禁止在本合同下悄悄迁移执行路径。

2.9.5 观察协议与不变量

另立 O2_SCHEMA version=1，以未门控全局周期记录，不让 O1 单在途检查器“放宽后”兼任 O2。每次父级接受产生唯一观察 seq，槽间转移携带 seq；seq 仅用于观察，不参与硬件功能。

最低事件：父级接受与等待（请求种类、选择、ready 及阻塞原因）、P_ACCEPT、P_TO_R、R_STALL、R_RESPONSE、P_KILL、R_KILL、RESET 丢弃、槽占用、DIV 开始/结束和重定向。按周期处理旧事务终结、移动和新接受，保留原始输出顺序，不假设 printf 先后决定硬件先后。

必须满足：

- 任意前缀：接受数 = 响应数 + 取消数 + reset 丢弃数 + 当前在途数；同一事务恰好属于一个终结类别。
- 当前在途数在 0..2；两个槽不能持有同一 seq；输出来自 R 中那个身份。
- 幸存输出按接受顺序，不允许重复/幽灵响应；完整 128 位及异常/写使能与独立模型一致。
- 多槽取消逐笔按 ROB 年龄判断，覆盖年龄顺序与接受顺序相反、flag 环绕、flushItself 两种情况。
- 反压保持整个 R 包稳定；只有匹配取消/reset 可以撤销旧响应。
- 父子握手一致，父级 out 与被选子输出一致，FU 输出 one-hot，门控不能冻结未终结请求。
- 正常运行结束无悬空请求；主动故障终止只能标部分检查，不能记为完整通过。

2.9.6 分阶段验收与负控

表 2-13 分阶段验收与负控

门禁	必须实际完成的内容	进入下一步的条件
G0 评估/合同	旧轨迹复算、两条接入路线审查、候选范围及停止条件	允许做隔离原型，不代表 O2 采用
G1 算术与弹性 FU	原 22+10000 完整 128 位向量；连续满管线、两槽反压/腾空；合法非法交错、v0 交错；部分/全部取消、reset 和环绕	无反压时真实 II=1 且最早响应 2 拍；事务守恒与反例检出通过
G2 真实父级	真 ExeUnit+VDot+VIDiv+VFDivSqrt+ClockGate；全部 FU 切换、满管线反压、cancel 与切换重合、持续供给及最终排空	无误派发/双完成/误关钟；其他 FU 受影响范围回归通过

表 2-13 分阶段验收与负控（续）

门禁	必须实际完成的内容	进入下一步的条件
G3 全核	新 O2 EMU、正常 NEMU；hello/vadd、117 唯一功能/异常矩阵、T12 等价控制压力；故障参考库及正常恢复	全部 DiffTest/完整状态通过，新观察事件覆盖达标
G4 公平性能	冻结相同 16 镜像、参考库、工具配置，O1/O2 各自实测 custom 与 RVV；同合同四场景和相同样本规则	有实际净收益、退化可解释、结果与身份绑定可复算
G5 采用与交付	O3 应用数据、O4 结构/时序证据、O5/T17 队友干净检出构建和人工复核	选择最终模式、冻结后再集中编写 T16/T18 材料

定向负控至少包括：交换 P/R 的 pdest 或 ROB；篡改高位结果；漏取消其中一槽；误杀另一个幸存槽；重复/丢失响应；R 反压时修改字段；父子握手不一致；双 FU 输出有效；有请求时误关钟。区分真实硬件故障注入与日志/检查器故障注入，不以后一种冒充前一种。

性能采用条件：先争取冻结 batch128 相对 O1 再降至少 10%（362.50→不高于 326.25 cycles），这是本合同新增的工程投入目标，不是预言、官方标准或正确性通过线；必须同时报告全部场景，其他保留场景均值退化超过 2% 先调查。若 RVV 保持 678，这个目标约对应 2.078×，正式加速比仍用候选自身实测 RVV。达到 2× 本身不自动通过采用门禁。

2.10 本轮已提供截图补充

本节收录已有补充截图，并按各自历史版本和证据层级解释。正式图位已补齐；尚未提供的原始波形或专门异常截图，其补证范围按正文保留。

(a) Fixed 0f VDotProducts source excerpt

```
class VDotProducts extends RawModule {
  val io = IO(new Bundle {
    val vs1 = Input(UInt(128.W))
    val vs2 = Input(UInt(128.W))
    val products = Output(Vec(8, SInt(16.W)))
  })

  for (lane <- 0 until 8) {
    val a = io.vs1(8 * lane + 7, 8 * lane).asSInt
    val b = io.vs2(8 * lane + 7, 8 * lane).asSInt
    io.products(lane) := a * b
  }
}

/** Balanced, widening signed reduction of eight 16-bit products.
  * Only the low 32 bits receive the sign-extended sum; bits 127:32 are zero.
  * No mask, tail, old destination, rounding, or saturation is involved.
  */
class VDotReduce extends RawModule {
```

(b) Fixed 0f VDotReduce source excerpt

```
/** Balanced, widening signed reduction of eight 16-bit products.
  * Only the low 32 bits receive the sign-extended sum; bits 127:32 are zero.
  * No mask, tail, old destination, rounding, or saturation is involved.
  */
class VDotReduce extends RawModule {
  val io = IO(new Bundle {
    val products = Input(Vec(8, SInt(16.W)))
    val result = Output(UInt(128.W))
  })

  private val sums17 = Wire(Vec(4, SInt(17.W)))
  private val sums18 = Wire(Vec(2, SInt(18.W)))
  private val sum19 = Wire(SInt(19.W))

  for (pair <- 0 until 4) {
    sums17(pair) := io.products(2 * pair) +& io.products(2 * pair + 1)
  }
  for (pair <- 0 until 2) {
    sums18(pair) := sums17(2 * pair) +& sums17(2 * pair + 1)
  }
  sum19 := sums18(0) +& sums18(1)

  private val result32 = Wire(SInt(32.W))
  result32 := sum19
  io.result := Cat(0.U(96.W), result32.asUInt)
}

/** Combinational arithmetic top for independent verification.
  * The future functional-unit wrapper instantiates the same two submodules
  * separately, with registers between them; this top has no cycle protocol.
  */
```

图 2-11　固定 0f 算术实现源码摘录

源代码截图辅助解释 8 路乘法和归约；不替代 S2-02 数据通路图。

Host model checks only; no RTL/NEMU execution in this command



图 2-12 主机端参考模型检查画面

35 个测试方法通过；该命令只检查主机模型和数据，不授 RTL 或 NEMU 资格。

Historical 0f observer log excerpt; NOT a GTKWave waveform

```
S2-05 SUPPORTING EVIDENCE: VDOT TWO-SLOT EVENT LOG EXCERPT
Historical 0f clean-02, dense-batch128, 2026-10-27 (UTC).
Observer event log excerpt; this is NOT a waveform screenshot.
Selected fields copied from the source lines listed below; fields omitted.
Source: /home/riscv/riscv-lab/software-import-20261026/clean-02/fullcore-run-01/runs/dense-batch128/run.log
Source SHA256: 18f472d75feef0378bec71c64793ffd2297bc37c47a5b501597f222da174b98f

line 146 cycle=10984 O2_STATE
  pvalid=0 pseq=0 rvalid=0 rseq=0 occupancy=0 invalid=1 inready=1 outvalid=0 outready=1 move=0
line 148 cycle=10984 O2_ACCEPT
  seq=0 rob=1:31 pdest=110 vl=8 illegal=0 vecwen=1
line 149 cycle=10985 O2_STATE
  pvalid=1 pseq=0 rvalid=0 rseq=0 occupancy=1 invalid=1 inready=1 outvalid=0 outready=1 move=1
line 151 cycle=10985 O2_ACCEPT
  seq=1 rob=1:32 pdest=111 vl=8 illegal=0 vecwen=1
line 152 cycle=10985 O2_MOVE
  seq=0 rob=1:31 pdest=110 illegal=0 vecwen=1
line 153 cycle=10986 O2_STATE
  pvalid=1 pseq=1 rvalid=1 rseq=0 occupancy=2 invalid=1 inready=1 outvalid=1 outready=1 move=1
line 155 cycle=10986 O2_ACCEPT
  seq=2 rob=1:33 pdest=112 vl=8 illegal=0 vecwen=1
line 156 cycle=10986 O2_MOVE
  seq=1 rob=1:32 pdest=111 illegal=0 vecwen=1
line 157 cycle=10986 O2_RESPONSE
  seq=0 rob=1:31 pdest=110 data=0x00000000000000000000000000020000 vecwen=1

At cycle 10986: ACCEPT seq=2, MOVE seq=1, RESPONSE seq=0.
Source result.json reports O2_DENSE_ELASTIC_AND_WINDOWS_PASS and exit_code=0.
Original batch UTC capacity audit FAIL retained; see R5/G5 acceptance RESULT.md.
```

图 2-13　两槽重叠事务日志摘录

保留 10986 周期 ACCEPT seq2、MOVE seq1、RESPONSE seq0；本图为历史日志字段摘录；S2-05 现为父级逐拍记录的事件重绘，未提供原始波形截图。原批次 UTC 容量审计 FAIL 保留。

(a) Slot registers and 32-bit result state

```
    require(!cfg.writeFflags && !cfg.writeVxsat && !cfg.flushPipe && !cfg.replayInst)

    val pipelineStatus = IO(Output(new VDotElasticStatus))

>   private val pValid = RegInit(false.B)
    private val rValid = RegInit(false.B)
    private val pProducts = Reg(Vec(8, SInt(16.W)))
    private val pCtrl = Reg(chiselTypeOf(io.out.bits.ctrl))
    private val pPerf = Reg(chiselTypeOf(io.out.bits.perfDebugInfo))
    // H1 stores only the architectural low word; the public result stays 128 bits.
    // Keep each slot's complete control/perf identity on the same transfer edge.
    private val rPacket = Reg(new Bundle {
      val ctrl = chiselTypeOf(io.out.bits.ctrl)
      val res = new Bundle {
        val data = UInt(32.W)
      }
      val perfDebugInfo = chiselTypeOf(io.out.bits.perfDebugInfo)
    })

    pipelineStatus.pValid := pValid
    pipelineStatus.rValid := rValid
    pipelineStatus.occupancy := pValid.asUInt +& rValid.asUInt

    private val illegalInput = vill || vsew =/= VSew.e8 || vlmul =/= VLmul.m1 ||
      !vm || vstart =/= 0.U || vl =/= 8.U || fuOpType =/= VDotOpType.s8x8
```

(b) Handshake and cancellation conditions

```
PS C:\WINDOWS\system32> wsl -d Ubuntu-22.04 -- git -C /home/riscv/riscv-lab/zjhwork2-review-20261018/repo --no-pager show 0f12df6753785
56670afd0dd2153f47fcb330610:src/main/scala/xiangshan/backend/fu/wrapper/VDotElastic.scala | Select-String -Pattern 'private val rReady
=' -Context 5,12

      !vm || vstart =/= 0.U || vl =/= 8.U || fuOpType =/= VDotOpType.s8x8
    private val killP = pValid && pCtrl.robIdx.needFlush(io.flush)
    private val killR = rValid && rPacket.ctrl.robIdx.needFlush(io.flush)
    private val liveP = pValid && !killP && !reset.asBool
    private val liveR = rValid && !killR && !reset.asBool
>   private val rReady = !liveR || io.out.ready
    private val move = liveP && rReady
    private val pReady = !liveP || rReady

    // Parent admission is enforced by its atomic child dispatch. The FU also
    // suppresses every new accept on redirect/reset, including unrelated flush.
    io.in.ready := pReady && !io.flush.valid && !reset.asBool
    io.out.valid := liveR
    io.out.bits.ctrl := rPacket.ctrl
    io.out.bits.perfDebugInfo := rPacket.perfDebugInfo
    io.out.bits.res := 0.U.asTypeOf(io.out.bits.res)
    io.out.bits.res.data := Cat(0.U(96.W), rPacket.res.data)
```

(c) State update and old-P transfer

```
PS C:\WINDOWS\system32>
                       > wsl -d Ubuntu-22.04 -- git -C /home/riscv/riscv-lab/zjhwork2-review-20261018/repo --no-pager show 0f12df6753785
56670afd0dd2153f47fcb330610:src/main/scala/xiangshan/backend/fu/wrapper/VDotElastic.scala | Select-String -SimpleMatch 'pValid := (live
P && !move) || io.in.fire' -Context 2,16

    acceptedCtrl.exceptionVec.get(ExceptionNO.illegalInstr) := illegalInput

>   pValid := (liveP && !move) || io.in.fire
    rValid := (liveR && !io.out.fire) || move

    when(io.in.fire) {
      pProducts := products.io.products
      pCtrl := acceptedCtrl
      pPerf := io.in.bits.perfDebugInfo
    }
    when(move) {
      // Reads pre-edge P even when a new request replaces it at this same edge.
      rPacket.ctrl := pCtrl
      rPacket.perfDebugInfo := pPerf
      rPacket.res.data := Mux(pCtrl.exceptionVec.get(ExceptionNO.illegalInstr),
        0.U(32.W), reduce.io.result(31, 0))
    }

    // Read-only handles for the separate G1/G2 observation harness. These carry
```

图 2-14　固定 0f 两槽状态与控制源码摘录

依次展示寄存器、握手与取消条件、状态更新；源代码画面辅助解释实现；S2-06 至 S2-08 的控制行为及证据范围以各图所列原始记录为准。